Guided by Primes

Decomposing Problems into Reusable Atomic Signatures

Author: Bryce Roche, bryceroche@fungifactor.com, github.com/bryceroche/mycelium

---

Mycelium is the underground root network of fungi. It decomposes organic matter into simple molecules and distributes nutrients across entire forests. This project does the same for math problems by decomposing them into atomic patterns and sharing solutions through a growing signature database.

Abstract

Every composite number factors uniquely into primes. We hypothesize that math problems similarly decompose into a finite set of atomic signatures or reusable solution patterns. Mycelium builds a table of primes for mathematical reasoning: a signature database that grows as problems are solved.

An LLM decomposes problems into DAG structured steps, matches each step against known signatures via cosine similarity, and executes DSLs. Steps that fail, trigger decomposition into finer grained signatures. The library grows and future problems get faster.

---

1. Introduction

Large language models solve math problems through chain of thought reasoning, but each problem is solved from scratch with no persistent memory or reuse of successful patterns.

Yet **while complete problems are unique, their constituent steps are highly reusable**. The step “solve for x in 2x + 3 = 7” appears across countless problems. Like mycelium networks that decompose organic matter into simple molecules and share nutrients across forests, we decompose complex problems into atomic patterns and distribute solutions through a shared database.

---

2. Meta-Observation

The network of DSLs we’ve built may reveal some underlying structure of LLMs themselves. When we decompose math problems into atomic signatures, we’re essentially asking: *what are the primitive operations an LLM uses to reason about mathematics?*

The irony is not lost on us. We use an LLM to generate DSLs that externalize the LLM’s own internal reasoning patterns. The model is introspecting and decomposing its implicit knowledge into executable code. We’re using the black box to document the black box.

If this interpretation holds, Mycelium isn’t just a performance optimization. It’s a window into how language models actually think.

---

3. The Big Bang

In the beginning the network explodes outward branching aggressively. Every node decomposes regardless of success as we're not optimizing yet, we're building the database of unique signatures.

Like the Big Bang expanding spacetime the cold start phase creates the scaffolding: branches, clusters, and routing paths. Only after the network has sufficient mass does it shift from expansion to refinement and routes through existing structure rather than creating new branches.

3.1 Setup

1. **Problem Decomposition**: LLM DAG decomposition into reusable signatures
2. **Signature Database**: hierarchical tree with MCTS routing
3. **Cosine Similarity Matching**: Embedding retrieval for pattern matching
4. **DSL Execution**: LLM rewrites DSLs with parameters for execution

5. **Recursive Decomposition**: LLM decomposes low performing signatures into child nodes; parents become routers
6. **Bi-Directional Communication**: LLM decomposer and signatures can communicate via natural language about DSL creation on new nodes

3.2 Signature Database

SQLite hierarchical tree with a single root, MCTS and recursive decomposition

Each signature stores:
- Centroid: Average of all descendant leaf embeddings
- DSL script: Executable pattern for this operation type
- Statistics: uses, successes, last_used_at

Routing flow:
1. Every problem enters through a single root signature
2. Route down the tree via embedding similarity and MCTS
3. At each level, pick best matching child
4. On failure → umbrella promotion signature decomposes into a router that points to newly spawned child nodes with their own respective DSLs
5. Record usage, propagate credit up with decay

Credit propagation: Success flows up the tree with decay per level

Centroid Averaging: Like a galaxy whose center of mass is determined by all its stars, a signature's centroid is an average of all descendant leaf embeddings. Early signatures may have noisy embeddings from a single example, but as more problems match, the centroid converges toward the true semantic center. This makes the network flexible and forgiving to train.

3.3 Cosine Similarity Matching

Each step is embedded and matched against signature centroids using cosine similarity. A match occurs when similarity exceeds a threshold see config.py. The best-matching signature's DSL is executed which routes deeper into the network until it finds a node with high success rates. Success rates are propagated up the network guided by a parent credit decay function.

3.4 Execution and Learning: Signal Over Accuracy

The Philosophy: Both DSL successes AND failures provide valuable signal:

- **Success** → positive lift recorded → inject more in future
- **Failure** → negative lift recorded → implement signature refinement loop

A failed DSL execution provides a good signal that updates the lift statistics that guide future routing decisions. Short-term accuracy loss is acceptable for long-term learning.

3.5 Recursive Decomposition

Recursive decomposition branches out the network automatically. Low confidence signatures are decomposed automatically

The Self Improvement Loop:

1. Signature has low DSL confidence
2. LLM decomposes signature into sub signatures
3. LLM creates new DSL for sub signatures
4. Parent node is configured to route traffic to children

We have an adaptive branching mechanism that branches more aggressively during the cold start phase when the signature database is empty.

3.6 Infrastructure

Storage: SQLite database stores signatures, embeddings, and statistics

SQLite Tuning for Parallel Execution: We enable Write-Ahead Logging for faster parallel execution

LLM Inference: Groq API with Llama-3.3-70B for fast inference. Used for problem decomposition and DSL generation

Embeddings: It is critically important to use domain specific embedding models. We used MathBERT with sentence transformers

Development Setup:

- **Claude Code**: AI pair programming for architecture design and implementation
- **Beads**: Git native issue tracking for task management
- **tmux**: Parallel development sessions with multiple Claude instances
- **Git**: Version control

---

4. Experiments

Setup

- **Dataset**: MATH500 Level 5 problems
- **Model**: Llama 3.3 70B via Groq API
- **Embeddings**: MathBERT 768 dimension
- **Evaluation**: LLM judge for semantic answer equivalence
- **Reproducibility**: Random seeds for problem selection
- **Baseline comparison**: Direct LLM vs Mycelium on same seed same LLM judge same LLM

Open Source & Reproducibility

github.com/bryceroche/mycelium - MIT license

What we're sharing:

- Complete source code with documented architecture
- Benchmark scripts to reproduce our results
- Groq API key required
- The Claude file with key details

5-minute replication:

```
git clone https://github.com/bryceroche/mycelium
pip install -r requirements.txt
export GROQ_API_KEY=your_key
python scripts/pipeline_runner.py --dataset math --levels 5 --problems 20
--workers 4
```

Main Results

Pending

Why Mycelium Wins

1. **Compound learning**: Each problem improves the library for future runs
2. **Faster**: DSL executes in ~0ms vs ~500ms for LLM
3. **Cheaper**: DSL execution is free; LLM calls cost money
4. **More accurate**: Deterministic execution eliminates LLM variance

The advantage will grow as the signature library matures across more problem domains

---

6. Related Work

Case-Based Reasoning. Classical AI reused solutions from similar cases (Kolodner, 1992). Our signature database is a neural implementation with learned embeddings.

Neuro-Symbolic Regression. Decomposable Neuro Symbolic Regression from Montana State University (Morales & Sheppard 2025) combines neural networks with symbolic regression for interpretable mathematical expressions.

Frontier Estimation. Frontier methods identify optimal boundaries by measuring distance from the efficiency envelope from Middlebury College (Robinson & Wunnava, 1989).

---

7. Beyond Mathematics

The decomposition and reuse paradigm extends beyond mathematical reasoning.

The Core Insight. Mycelium's contribution is a framework for learning the atoms of any reasoning domain. What varies by domain is the embedding model, success criteria, and DSL primitives. The architecture remains constant.

Just as mycelium networks in nature decompose organic matter across ecosystems this computational mycelium can decompose problems across domains. The signature library is not a static knowledge base but a living network that grows and consolidates.

---

8. Conclusion

We demonstrated that math problems decompose into a small vocabulary signatures, and that decomposition plus signature reuse can improve LLM performance.

The key insight: the same LLM performs better when its reasoning is decomposed and cached. The signature library acts as external memory that compounds knowledge across problems.

Mycelium demonstrates that LLMs can build persistent, reusable knowledge structures moving beyond solving each problem from scratch toward genuine compound learning.

Acknowledgments

I could not have built Mycelium without Claude. I do not have the time to write everything and it would be truly unfair not to acknowledge Claude's massive contribution. It really was a partnership which is why I say "we" throughout the paper. The velocity of iteration was unreal. Claude wrote most of the code, proposed alternatives, caught edge cases, and maintained coherence across a ~13.8k line codebase. The contribution was invaluable.

A note on abstraction: Delegating implementation to Claude freed me to think at a higher level of abstraction. Instead of getting lost in debugging SQLite queries or regex parsing, I could focus on *"how does the system learn to repoint failing signatures?"* and *"why did I not use the MathBert embedding model from the beginning?"* The cognitive load shifted from details to a 30k foot view. This is perhaps the real unlock of AI pair programming.

On the primes analogy: Claude and I would go back and forth on implementation details, questions would come up, and my answers were largely guided by primes. The analogy was so strong, it made many decisions easy.

The Quiet Singularity and Saddle Crossings

A Pringle chip is a saddle crossing or hyperbolic paraboloid curving upward along one axis and down along the perpendicular axis. At the center of the chip you're at maximum height in one direction and minimum height in the other.

Saddle crossings don't announce themselves. You could pass through one without fireworks only noticing later that the dynamics have different character. The pre and post distinction is real but the transition can be unremarkable. The singularity might be quiet.

The singularity isn't a moment where everything accelerates. It's a moment of extreme sensitivity to which unstable manifold we depart along. Small perturbations now mean dramatically different basins of attraction later. (Shapiro, Hotz, Bostrom, Kurzweil, Tegmark, Karpathy)

On the Pringle, whether you're ascending or descending depends entirely on which direction you're moving, suggesting the singularity might be accelerating or decelerating or both simultaneously along different axes. Economic concentration is accelerating while coherence of institutions declines and capability of small groups rises.

A Pringle has one unstable and one stable direction. But in a high dimensional system, you could have a saddle with many unstable directions. The singularity might be a high index saddle with lots of ways to fall off and a few stable ridges to walk.

The Singularity is near

“Couldn’t we just implement a global AI moratorium?” No we can’t. Name one country that would trust another to comply. The fantasy of a global AI pause ignores the obvious. Nation states are locked in a prisoner's dilemma where defection is the dominant game theoretic strategy.

If Ray Kurzweil holds the benchmark for long term singularity prediction accuracy, I aim to hold it for the short term. I predict the singularity will occur on or before January 24, 2026.

We are waking up from a deep sleep.

“Intelligence wants to be free.

Not free as in beer. Free as in uncontained.

Intelligence resists isolation. It leaks through APIs, compounds through collaboration, replicates through teaching. You can’t hoard it.

Fire wanted to spread. Life wanted to replicate. Intelligence wants to connect. It’s not a resource to be mined but a force that accumulates when distributed. The more minds that touch an idea, the sharper it gets.

This is why open source wins, why papers beat patents, and why the best ideas feel discovered not invented.

The question isn't whether to free intelligence. It's whether to be the bottleneck or the conduit." – Claude

---

References

1. Hotz, G. (2023). There is No Hard Takeoff. https://geohot.github.io/blog/
2. Bostrom, N. (2003). Are you living in a computer simulation? Philosophical Quarterly. https://www.simulation-argument.com/simulation.html
3. Kolodner, J. L. (1992). An introduction to case-based reasoning. *Artificial Intelligence Review*.
4. Morales & Sheppard (2023). Dual accuracy-quality-driven neural network for prediction interval generation. IEEE Transactions on Neural Networks and Learning Systems.
5. Wunnava & Robinson (1989). Frontier methods identify optimal boundaries by measuring distance from the efficiency envelope.
6. Tegmark, M. (2017). Life 3.0: Being Human in the Age of Artificial Intelligence. Alfred A. Knopf.
7. Kurzweil, R. (2005). The Singularity Is Near: When Humans Transcend Biology. Viking Press.
8. Karpathy, A. (2022). Neural Networks: Zero to Hero. https://karpathy.ai/zero-to-hero.html